

영상처리를 활용한 주차 차단기

2026-1학기 국립부경대학교 IoT 시스템 프로그래밍 기말 프로젝트

202313431 임승호 (순번: 19)

목차

- 개요

- 프로젝트 기획
- 전체적인 설계안
- 외부 라이브러리 및 기술

- 진행 과정

- 외부 라이브러리 설치
- yolov5를 이용한 번호판 인식 학습
- 프로그램 설계 및 구현

- 코드 상세

- 센서(sencor.cpp)
- 게이트(gate.cpp)
- LCD(lcd.cpp)
- 카메라(camera.cpp)
- 영상처리(ocr.cpp)
- main.cpp

- 마무리

- 프로젝트 시연
- 개선해야 할 사항
- 참고자료
- Q&A

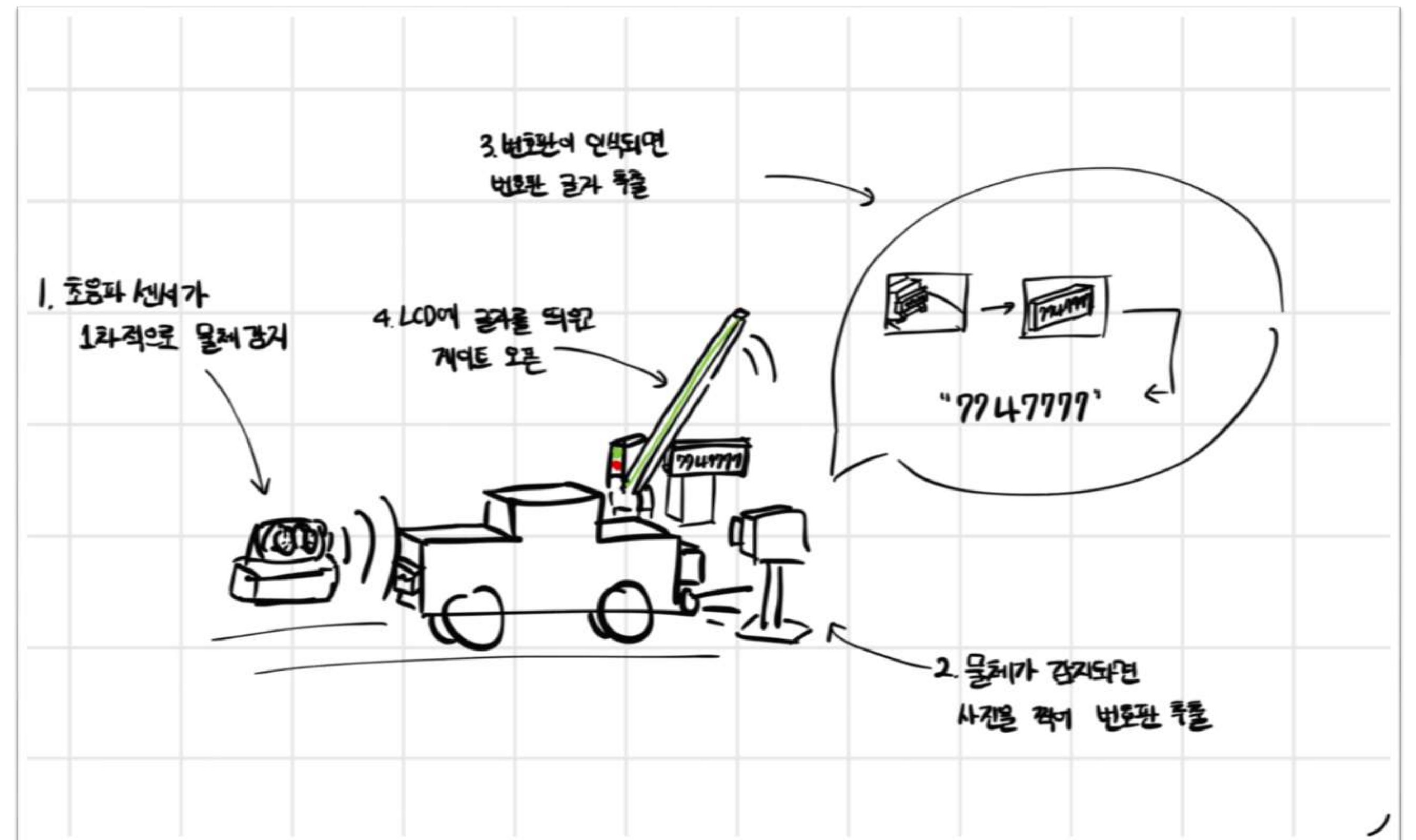
프로젝트 기획

• 기획 의도

- 일반적인 센서 뿐만 아닌, 카메라를 효과적으로 활용할 수 있는 프로젝트를 진행하고자 하였음.
- 사진 내 물체뿐 아닌 텍스트를 인식하는 프로젝트를 하면 어떨까 생각함
- 이에 해당하는 대표적인 예시가 "주차 차단기"

• 전체적인 구상

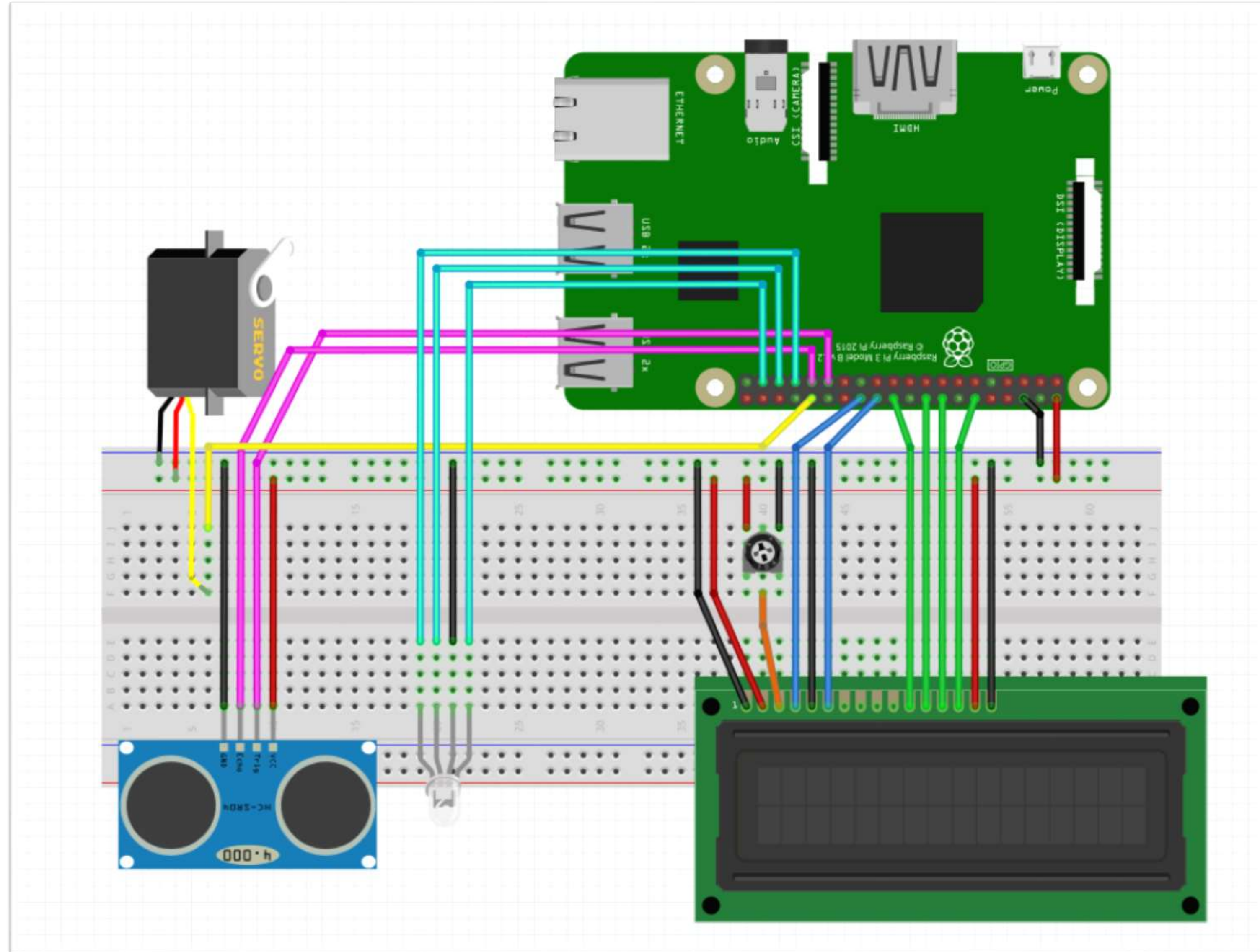
- 본래 주차 차단기는 루프 코일 센서를 활용하지만, 비싸기 때문에(...) 초음파 센서로 대체
- 차단기 모터는 실습실 servo 모터(SG90)를 사용
- 실습실 LCD는 영어 전용이기 때문에 한글 번호판에 나오는 글자 종류별로 직접 영어로 매핑 (ex. 가 = AA)
- 게이트에 추가적으로 RGB LED 사용



전체적인 설계안

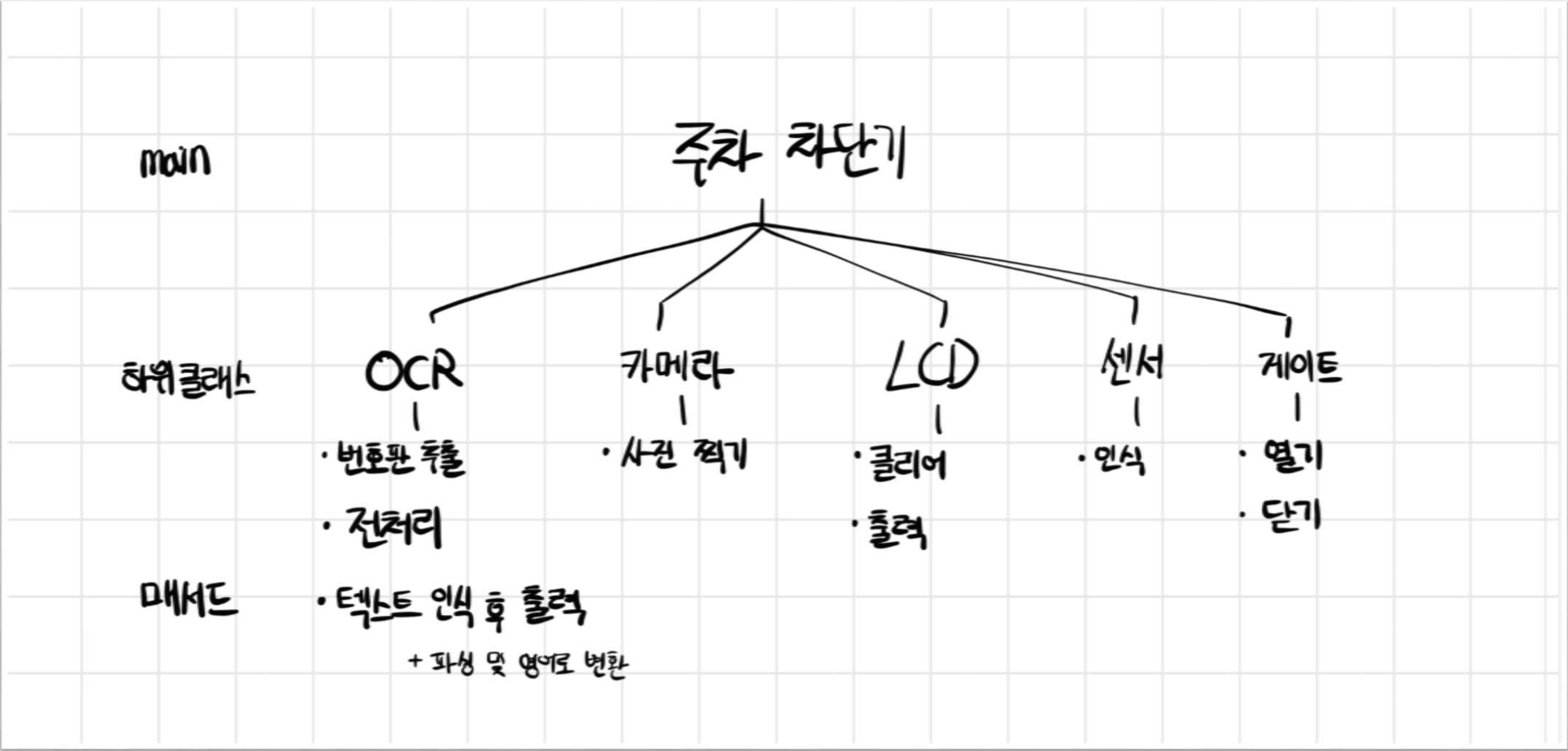
• 회로도

- RGB LED: GPIO 25(Red), 24(Green), 23(Blue)
- LCD: GPIO 1, 4, 5, 6, 7, 10, 11 (D7~D4, E, RS)
- 초음파 센서: GPIO 21(TRIG), 22(ECHO)
- Servo 모터: GPIO 26
- 카메라: usb 연결 (화면상엔 없음)



전체적인 설계안

- 클래스 설계



외부 라이브러리 및 기술

- **Opencv**: 이미지를 분석 및 처리하는 라이브러리. 카메라 사용 및 이미지 전처리에 사용
 - **dnn 모듈**: 학습한 모델을 불러와 추론하도록 설계된 딥러닝 라이브러리. 사진에서 번호판을 추출하는데 사용
- **tesseract**: 오픈소스 광학 문자 인식(OCR) 엔진 라이브러리. 번호판 텍스트 추출에 사용
 - **leptonica**: 이미지 처리 및 분석을 위한 라이브러리
- **wiringPi**: gpio 핀을 제어할 수 있도록 하는 라이브러리. 센서 및 게이트에 사용
- **yolov5**: 객체 탐지 딥러닝 모델. 사진에서 번호판을 인식하는 모델 학습에 사용
- **CMake**: 외부 라이브러리 및 프로젝트 빌드에 사용

외부 라이브러리 설치

- Opencv 4.14.0 버전 설치

※ 이전에는 4.8.1 버전을 사용하였으나, dnn 모듈이 학습 모델 파일인 .onnx의 커널 사이즈를 인식하지 못하여 4.14.0 버전으로 재설치

- 우선 opencv 빌드에 사용할 패키지들을 설치한다. (CMake 포함)
- opencv 공식 깃허브에 있는 opencv와 opencv_contrib을 라즈베리파이에 git clone
 - ※ <https://github.com/opencv/opencv> 및 https://github.com/opencv/opencv_contrib
- CMake를 통하여 빌드 후 설치
- 빌드 중 메모리가 모자랄 경우 "/etc/dphys-swapfile" 파일을 수정하여 가상 메모리를 늘림

```
● pi@rasberrypi:~ $ mkdir opencv
● pi@rasberrypi:~ $ cd opencv
● pi@rasberrypi:~/opencv $ mkdir build
● pi@rasberrypi:~/opencv/build $ cmake -D CMAKE_BUILD_TYPE=RELEASE -D CMAKE_INSTALL_PREFIX=/usr/local -D WITH_TBB=OFF -D WITH_IPP=OFF
-D WITH_1394=OFF -D BUILD_WITH_DEBUG_INFO=OFF -D BUILD_DOCS=OFF -D INSTALL_C_EXAMPLES=ON -D INSTALL_PYTHON_EXAMPLES=ON -D BUILD_E
AMPLES=OFF -D BUILD_TESTS=OFF -D BUILD_PERF_TESTS=OFF -D ENABLE_NEON=ON -D ENABLE_VFPV3=OFF -D WITH_QT=OFF -D WITH_GTK=ON -D WITH_
REETYPE=ON -D WITH_OPENGL=ON -D OPENCV_ENABLE_NONFREE=ON -D OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib/modules -D WITH_V4L=ON
-D WITH_FFMPEG=ON -D WITH_XINE=ON -D ENABLE_PRECOMPILED_HEADERS=OFF -D BUILD_NEW_PYTHON_SUPPORT=ON -D OPENCV_GENERATE_PKGCONFIG=ON
-D PYTHON3_PACKAGES_PATH=/usr/local/lib/python3.13/dist-packages ..
```

```
● pi@rasberrypi:~/opencv/build $ time make -j4
```

```
● pi@rasberrypi:~/opencv/build $ sudo make install
```


외부 라이브러리 설치

- tesseract 및 leptonica 설치
 - opencv와 같이 tesseract를 clone하여 빌드하여도 되지만, "sudo apt-get install"로 설치하여도 무방
 - leptonica의 경우도 같은 방식으로 설치
- wiringPi의 경우는 강의 시간에 설치하였던 대로 사용
- yolov5는 라즈베리파이가 아닌 개인용 PC에 설치 후 모델 학습용으로 사용

yolov5를 이용한 번호판 인식 학습

- 설치 및 학습 (windows11 기준)

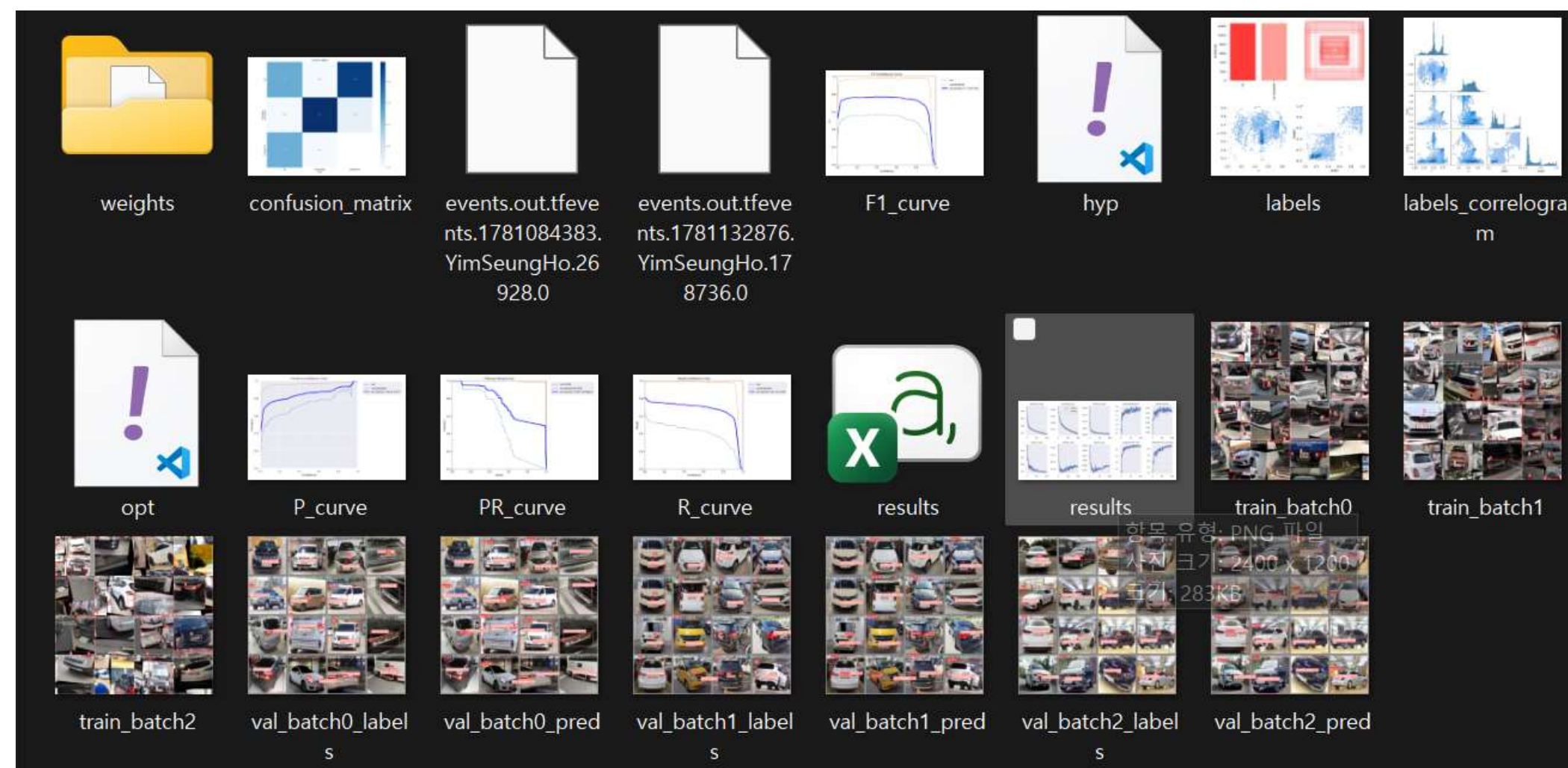
- Pytorch를 사용해야 하므로 anaconda라는 프로그램을 사용하여 가상환경을 구축하여 학습

- yolov5는 깃허브에 있는 파일을 clone하여 설치

- ※ <https://github.com/ultralytics/yolov5>

- 데이터 셋은 roboflow라는 사이트에 있는 공개 데이터 셋을 사용

- ※ <https://universe.roboflow.com/school-b6rj0/-dpo7a/browse>



```
(base) C:\Users\shoot>conda activate yolov5
```

```
(yolov5) C:\Users\shoot>cd yolov5-master
```

```
(yolov5) C:\Users\shoot\yolov5-master>python train.py --img 620 --batch 16 --epochs 100  
--data ./dataset/data.yaml --cfg ./models/yolov5s.yaml --weights yolov5s.pt
```

Epoch	0/99	gpu_mem	1.48G	box	0.1081	obj	0.02775	cls	0.04739	labels	69	img_size	416: 100%	55/55	[00:14:00:00, 3.69it/s]
		Class	all	Images	250	Labels	454	P	0.013	R	0.129	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 4.87it/s]
									0.0102		0.00922				
Epoch	1/99	gpu_mem	1.85G	box	0.07821	obj	0.08125	cls	0.04042	labels	47	img_size	416: 100%	55/55	[00:12:00:00, 4.49it/s]
		Class	all	Images	250	Labels	454	P	0.212	R	0.563	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 4.71it/s]
									0.0219		0.0872				
Epoch	2/99	gpu_mem	1.85G	box	0.06298	obj	0.02793	cls	0.03719	labels	71	img_size	416: 100%	55/55	[00:12:00:00, 4.50it/s]
		Class	all	Images	250	Labels	454	P	0.308	R	0.514	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 5.38it/s]
									0.297		0.154				
Epoch	3/99	gpu_mem	1.85G	box	0.06355	obj	0.0281	cls	0.03286	labels	57	img_size	416: 100%	55/55	[00:12:00:00, 4.57it/s]
		Class	all	Images	250	Labels	454	P	0.279	R	0.581	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 5.23it/s]
									0.358		0.185				
Epoch	4/99	gpu_mem	1.85G	box	0.06841	obj	0.02534	cls	0.02821	labels	61	img_size	416: 100%	55/55	[00:12:00:00, 4.55it/s]
		Class	all	Images	250	Labels	454	P	0.295	R	0.618	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 5.57it/s]
									0.425		0.228				
Epoch	5/99	gpu_mem	1.85G	box	0.06396	obj	0.02346	cls	0.02556	labels	61	img_size	416: 100%	55/55	[00:12:00:00, 4.54it/s]
		Class	all	Images	250	Labels	454	P	0.307	R	0.603	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 5.31it/s]
									0.433		0.238				
Epoch	6/99	gpu_mem	1.85G	box	0.05168	obj	0.02224	cls	0.02216	labels	67	img_size	416: 100%	55/55	[00:12:00:00, 4.45it/s]
		Class	all	Images	250	Labels	454	P	0.414	R	0.556	mAP@.5	mAP@.5: .95: 100%	8/8	[00:01:00:00, 5.71it/s]
									0.556		0.289				

yolov5를 이용한 번호판 인식 학습

- .pt를 onnx로 변환

- 학습된 모델인 .pt파일을 c++ 프로젝트에서 사용하기 위해서는 .onnx파일로 변환을 해주어야 함.
- yolov5에 있는 export.py를 실행하여 .pt파일을 .onnx파일로 변환.
- 변환한 .onnx 파일을 라즈베리파이로 이식해준다.

```
(yolov5) C:\Users\shoot\yolov5-master>python export.py --weights ./runs/train/result/weights/best.pt  
--include onnx --img 640 --batch 1
```

이름	수정한 날짜	유형	크기
 best.onnx	2026-06-11 오후 6:15	ONNX 파일	28,131KB
 best.onnx.data	2026-06-11 오후 6:14	DATA 파일	27,840KB
 best.pt	2026-06-11 오전 4:30	PT 파일	14,110KB
 last.pt	2026-06-11 오전 4:30	PT 파일	14,110KB
 transfer	2026-06-11 오전 10:43	Python 원본 파일	2KB

프로그램 설계 및 구현

- assets

- yolov5로 번호판 개체를 감지하는 모듈
- 프로그램 실행 과정에서 찍은 사진 및 전처리 과정에서의 사진을 저장

- include

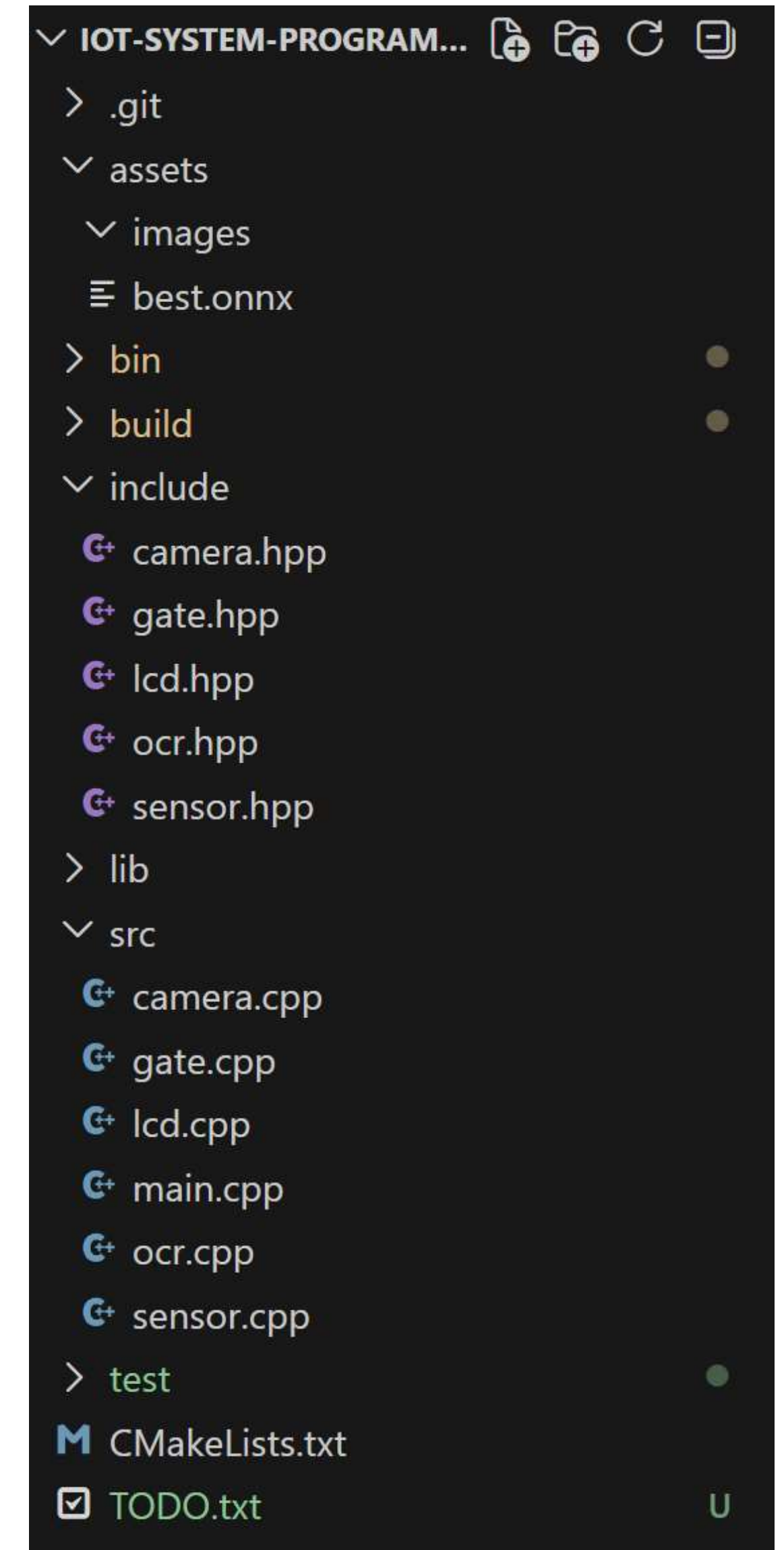
- 프로그램 구현에 필요한 사용자 정의 헤더 파일

- src

- 사용자 정의 헤더 파일에 선언된 각 객체들의 method
- 메인 프로그램 (main.cpp)

- 기타

- bin: 컴파일된 프로그램을 저장하는 파일
- test: 객체들을 만들면서 테스트한 테스트 코드들을 저장
- lib: 외부 라이브러리 저장 (opencv, wiringPi 등..)



sensor

```
3  #define TRIG_PIN 21
4  #define ECHO_PIN 22
5  #define LOW_RANGE_DIST 3
6  #define HIGH_RANGE_DIST 100
7
8  Sensor::Sensor() {
9      pinMode(TRIG_PIN, OUTPUT);
10     pinMode(ECHO_PIN, INPUT);
11     digitalWrite(TRIG_PIN, LOW);
12     std::cout << "Sensor is created" << std::endl;
13 }
14
15 Sensor::~~Sensor() {
16     digitalWrite(TRIG_PIN, LOW);
17     std::cout << "Sensor is destroyed" << std::endl;
18 }
```


sensor::detectObject

```
20 bool Sensor::detectObject() {
21     long long begin_time, start_time, end_time;
22
23     // begin time: 초음파 발사 시간
24     begin_time = micros();
25     digitalWrite(TRIG_PIN, HIGH);
26     delayMicroseconds(10);
27     digitalWrite(TRIG_PIN, LOW);
28
29     // start time: ECHO가 LOW->HIGH를 탐지한 최초 시간
30     start_time = micros();
31     while (digitalRead(ECHO_PIN) == LOW &&
32           ((micros() - begin_time + UINT32_MAX) % UINT32_MAX < 1e6))
33         start_time = micros();
34
35     // end time: ECHO가 HIGH->LOW를 탐지한 최초 시간
36     end_time = micros();
37     while (digitalRead(ECHO_PIN) == HIGH &&
38           ((micros() - begin_time + UINT32_MAX) % UINT32_MAX < 1e6))
39         end_time = micros();
40
41     // 거리 계산 (단위: cm)
42     long double difference = (end_time - start_time + UINT32_MAX) % UINT32_MAX;
43     long double distance = 0.03465 * (difference / 2.0);
44
45     // 3 ~ 100 cm 범위 이내로 측정 시 TRUE return
46     if (distance < LOW_RANGE_DIST || HIGH_RANGE_DIST < distance) {
47         std::cout << "Out of Range" << std::endl;
48         return false;
49     } else {
50         std::cout << "Distance : " << distance << "cm" << std::endl;
51         return true;
52     }
53 }
```


gate

```
2
3  #define RED_PIN 25
4  #define GREEN_PIN 24
5  #define BLUE_PIN 23
6  #define MOTER_PIN 26
7
8  Gate::Gate() {
9      softPwmCreate(MOTER_PIN, 0, 200);
10     pinMode(RED_PIN, OUTPUT);
11     pinMode(GREEN_PIN, OUTPUT);
12     pinMode(BLUE_PIN, OUTPUT);
13     close();
14     std::cout << "Gate is created" << std::endl;
15
16 }
17
18 Gate::~~Gate() {
19     setRGB(0, 0, 0);
20     std::cout << "Gate is destroyed" << std::endl;
21 }
22
```

```
23 void Gate::setRGB(bool red, bool green, bool blue) {
24     digitalWrite(RED_PIN, red);
25     digitalWrite(GREEN_PIN, green);
26     digitalWrite(BLUE_PIN, blue);
27 }
28
29 bool Gate::open() {
30     std::cout << "게이트가 열렸습니다." << std::endl;
31     setRGB(0, 1, 0);
32     softPwmWrite(MOTER_PIN, 25);
33     delay(1000);
34     return true;
35 }
36
37 bool Gate::close() {
38     std::cout << "게이트가 닫혔습니다." << std::endl;
39     setRGB(1, 0, 0);
40     softPwmWrite(MOTER_PIN, 15);
41     delay(1000);
42     return true;
43 }
```


lcd

```
3  #define LCD_RS 11
4  #define LCD_E 10
5  #define LCD_D4 6
6  #define LCD_D5 5
7  #define LCD_D6 4
8  #define LCD_D7 1
9
10 Lcd::Lcd() {
11     std::cout << "LCD is created" << std::endl;
12     lcdState = lcdInit(2, 16, 4, LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7, 0,
13     | | | | | | | | | | 0, 0, 0);
14     if (lcdState) std::cerr << "LCD init is failed" << std::endl;
15
16     lcdPosition(lcdState, 0, 0);
17 }
18
19 Lcd::~~Lcd() {
20     lcdClear(lcdState);
21     std::cout << "LCD is destroyed" << std::endl;
22 }
23
24 void Lcd::clear() {
25     lcdClear(lcdState);
26     std::cout << "LCD is cleared" << std::endl;
27 }
28
29 void Lcd::print(const std::string& str) {
30     lcdPosition(lcdState, 0, 0);
31     lcdPuts(lcdState, str.c_str());
32     std::cout << "LCD output: " << str << std::endl;
33 }
```


camera

```
3  bool Camera::openCamera(int index) {
4      cam.open(index);
5      return cam.isOpened();
6  }
7
8  cv::Mat Camera::capture() {
9      if (!cam.isOpened()) {
10         std::cerr << "카메라가 열리지 않았습니다" << std::endl;
11         return cv::Mat();
12     }
13
14     cv::Mat frame;
15     cam >> frame;
16     cv::imwrite("../assets/images/cam.jpg", frame);
17     std::cout << "카메라 사진 촬영 완료" << std::endl;
18     return frame;
19 }
```


ocr::cutting

```
67 cv::Mat Ocr::cutting(const cv::Mat& src) {
68     if (src.empty()) {
69         std::cerr << "이미지를 불러올 수 없습니다." << std::endl;
70         return src;
71     }
72
73     if (net.empty()) {
74         std::cerr << "모델이 선택되지 않았습니다." << std::endl;
75         return src;
76     }
77
78     // 원본 이미지의 복사본 생성
79     cv::Mat copy = src.clone();
80
81     // 라즈베리파이4 ARM CPU 명령어 최적화 가속
82     net.setPreferableBackend(cv::dnn::DNN_BACKEND_OPENCV);
83     net.setPreferableTarget(cv::dnn::DNN_TARGET_CPU);
84
85     cv::Mat blob;
86     cv::dnn::blobFromImage(src, blob, 1.0 / 255.0, cv::Size(IMAGE_SIZE, IMAGE_SIZE),
87         cv::Scalar(), true, false);
88     net.setInput(blob);
89
90     // 모델 추론
91     std::vector<cv::Mat> outputs;
92     net.forward(outputs, net.getUnconnectedOutLayersNames());
93     cv::Mat output(outputs[0].size[1], outputs[0].size[2], CV_32F,
94         outputs[0].ptr<float>());
95
96     // 결과 행렬의 행의 개수와 열의 길이
97     int rows = output.rows;
98     int dimensions = output.cols;
99
100    // 640 x 640 사진과 원본 사진의 비율 계산
101    float x_factor = src.cols / IMAGE_SIZE;
102    float y_factor = src.rows / IMAGE_SIZE;
103    float* data = (float*)output.data;
104
105    // 영역을 추려서 번호판에 대한 영역 및 신뢰도 저장
106    std::vector<cv::Rect> boxes;
107    std::vector<float> confidences;
108    for (int i = 0; i < rows; i++, data += dimensions) {
109        float objectness = data[4]; // data[4]: 사물 존재 확률 (Objectness)
110        if (objectness < SCORE_THRESHOLD) continue;
111
```

```
112    // 차, 번호판이 가능한 영역 (data[5], data[6]) 중에서 최대 확률을 가지는 영역을 추출
113    float* classes_scores = data + 5;
114    cv::Mat scores(1, 2, CV_32F, classes_scores);
115    cv::Point class_id_point;
116    double max_class_score;
117    minMaxLoc(scores, 0, &max_class_score, 0, &class_id_point);
118
119    // 영역이 가리키는 개체가 번호판이면서, 신뢰도 50% 초과면 영역을 저장
120    float total_score = objectness * max_class_score;
121    if (total_score > SCORE_THRESHOLD && class_id_point.x == 1) {
122        // [중심X, 중심Y, 가로폭, 세로폭]
123        float cx = data[0], cy = data[1], w = data[2], h = data[3];
124
125        // 좌상단 기준으로 좌표 변환 후 원본 이미지 스케일로 복원
126        int left = int((cx - 0.5 * w) * x_factor);
127        int top = int((cy - 0.5 * h) * y_factor);
128        int width = int(w * x_factor);
129        int height = int(h * y_factor);
130
131        // 영역, 신뢰도 저장
132        boxes.push_back(cv::Rect(left, top, width, height));
133        confidences.push_back(total_score);
134    }
135    }
136
137    // 중복된 영역 압축
138    std::vector<int> idxs;
139    cv::dnn::NMSBoxes(boxes, confidences, SCORE_THRESHOLD, NMS_THRESHOLD, idxs);
140    std::cout << "감지 번호판 개수 : " << idxs.size() << std::endl;
141
142    // 번호판 return (여러개 있을 경우 아래쪽에 있는 걸 우선)
143    cv::Mat cropped;
144    for (const int& idx : idxs) {
145        cv::Rect box = boxes[idx];
146        box = box & cv::Rect(0, 0, copy.cols, copy.rows);
147        if (!box.empty()) {
148            cropped = src(box);
149            bool success = imwrite("../assets/images/cutted.jpg", cropped);
150            if (success) std::cout << "번호판 저장 성공" << std::endl;
151        }
152    }
153    return cropped;
154 }
```


ocr::preprocessing

```
158 cv::Mat Ocr::preprocessing(const cv::Mat& src) {
159     if (src.empty()) {
160         std::cerr << "이미지를 불러올 수 없습니다." << std::endl;
161         return src;
162     }
163
164     cv::Mat resized, gray, blurred, binary, result;
165
166     cv::resize(src, resized, cv::Size(), 4.0, 4.0,
167         ||| ||| cv::INTER_CUBIC); // 사이즈 변환
168     cv::imwrite("../assets/images/resized.jpg", resized);
169
170     cv::cvtColor(resized, gray, cv::COLOR_BGR2GRAY); // 흑백 변환
171     cv::imwrite("../assets/images/gray.jpg", gray);
172
173     cv::GaussianBlur(gray, blurred, cv::Size(3, 3), 0); // 가우시안 블러
174     cv::imwrite("../assets/images/blurred.jpg", blurred);
175
176     cv::adaptiveThreshold(blurred, binary, 255, cv::ADAPTIVE_THRESH_GAUSSIAN_C,
177         ||| ||| ||| ||| ||| ||| cv::THRESH_BINARY, 99, 5); // 이진화
178     cv::imwrite("../assets/images/binary.jpg", binary);
179
180     cv::Mat kernel = cv::getStructuringElement(cv::MORPH_RECT, cv::Size(2, 2));
181     cv::morphologyEx(binary, result, cv::MORPH_OPEN, kernel); // 외곽선 정리
182     cv::imwrite("../assets/images/result.jpg", result);
183
184     std::cout << "전처리 완료" << std::endl;
185     return result;
186 }
```


ocr::changeText

```
186  std::string Ocr::changeText(const cv::Mat& src) {
187      if (src.empty()) {
188          std::cerr << "이미지를 불러올 수 없습니다." << std::endl;
189          return "";
190      }
191
192      tesseract::TessBaseAPI* ocrEngine = new tesseract::TessBaseAPI();
193      if (ocrEngine->Init(nullptr, "kor", tesseract::OEM_DEFAULT)) {
194          std::cerr << "Tesseract OCR 엔진을 초기화할 수 없습니다." << std::endl;
195          return "";
196      }
197
198      ocrEngine->SetImage(src.data, src.cols, src.rows, 1, src.step);
199      std::string result = ocrEngine->GetUTF8Text();
200
201      std::cout << "===== 텍스트 인식 결과 =====\n";
202      std::cout << result;
203      std::cout << "===== " << std::endl;
204
205      ocrEngine->End();
206      delete ocrEngine;
207      return result;
208  }
```


ocr::parsing

```
210 std::string Ocr::parsing(const std::string &str) {
211     int idx = 1e9;
212     std::string kor_to_rng;
213     for (int i = 0; i < (int)(str.size()); i++) {
214         if ((unsigned char)str[i] >= 0xEA && (unsigned char)str[i] <= 0xED) {
215             if (i + 2 < str.length()) {
216                 unsigned char b2 = (unsigned char)str[i + 1];
217                 unsigned char b3 = (unsigned char)str[i + 2];
218
219                 // '가'(0xEA 0x80 0x80) ~ '힉'(0xED 0x9E 0xBF) 범위 체크
220                 if (b2 >= 0x80 && b2 <= 0xBF && b3 >= 0x80 && b3 <= 0xBF) {
221                     idx = i, kor_to_rng = mapping[str.substr(i, 3)];
222                     break;
223                 }
224             }
225         }
226     }
227
228     if (idx == 1e9) return std::string();
229
230     std::string result;
231     for (int i = idx - 1, cnt = 0; i >= 0; i--) {
232         if (isdigit(str[i]) && cnt < 3) result.push_back(str[i]), cnt++;
233     }
234     reverse(result.begin(), result.end());
235     result += kor_to_rng;
236     for (int i = idx + 3, cnt = 0; i < (int)(str.size()); i++) {
237         if (isdigit(str[i]) && cnt < 4) result.push_back(str[i]), cnt++;
238     }
239
240     return result;
241 }
```


main

```
1  #include <csignal>
2
3  #include "../include/camera.hpp"
4  #include "../include/gate.hpp"
5  #include "../include/lcd.hpp"
6  #include "../include/ocr.hpp"
7  #include "../include/sensor.hpp"
8
9  Camera* camera = nullptr;
10 Gate* gate = nullptr;
11 Lcd* lcd = nullptr;
12 Ocr* ocr = nullptr;
13 Sensor* sensor = nullptr;
14
15 void sigHandler(int signum) {
16     std::cout << "Signal(" << signum << ") 감지됨." << std::endl;
17     if (camera != nullptr) delete camera;
18     if (gate != nullptr) delete gate;
19     if (lcd != nullptr) delete lcd;
20     if (ocr != nullptr) delete ocr;
21     if (sensor != nullptr) delete sensor;
22     std::cout << "Exit";
23     exit(0);
24 }
25
```

```
26 int main() {
27     if (wiringPiSetup() == -1) {
28         std::cerr << "wiringPiSetup 에러" << std::endl;
29         return -1;
30     }
31
32     // 객체들 생성
33     std::signal(SIGINT, sigHandler);
34     camera = new Camera;
35     gate = new Gate;
36     lcd = new Lcd;
37     ocr = new Ocr;
38     sensor = new Sensor;
39
40     camera->openCamera(0);
41     ocr->modelInit("../assets/best.onnx");
42     while (1) {
43         if (sensor->detectObject()) {
44             cv::Mat cropped = ocr->cutting(camera->capture());
45             if (!cropped.empty()) {
46                 cv::Mat pre = ocr->preprocessing(cropped);
47                 std::string plate_number = ocr->parsing(ocr->changeText(pre));
48                 lcd->print(plate_number);
49                 gate->open();
50
51                 while (sensor->detectObject()) delay(1000);
52                 delay(1000);
53                 lcd->clear();
54                 gate->close();
55             }
56         } else {
57             delay(1000);
58         }
59     }
60     return 0;
61 }
```


프로젝트 시연

- <https://youtu.be/IVkf7KbGIlg>



개선해야 할 사항

- tesseract 글자 학습으로 번호판 글자 인식률 높이기
 - tesseract-ocr 엔진을 학습시켜 특정 폰트에 대한 인식을 최적화 시킬 수 있음
- 번호판 사진 전처리 개선
 - counter 검출 및 허프 변환을 통해 번호판을 수평으로 맞추어 전처리하는 것이 가능
- servo 모터 제어 안정화
 - 전용 라이브러리인 Servo.h를 설치하여 pwm으로 조작하는 방식에 비해 안정적으로 조작 가능

참고자료

- 라이브러리 설치

- <https://makeutil.tistory.com/262> (opencv 설치)
- <https://forums.raspberrypi.com/viewtopic.php?t=370044> (메모리 스왑 기능)
- gemini (설치 도중 오류 해결)

- yolov5 학습

- <https://gentleman-blog.tistory.com/m/20> (yolov5 모델 사용법)

- 라이브러리 사용

- <https://dev-drive.tistory.com/34> (dnn 모듈을 사용한 객체 검출 방법)
- <https://inmile.tistory.com/8> (pwm을 사용한 servo 모터 조작 방법)
- <https://opencv-opencv.mintlify.app/examples/read-display> (opencv 라이브러리 기본 사용방법)
- <https://m.blog.naver.com/tommybee/221307497468> (tesseract 엔진 사용방법 예제)

Q&A